

Miscellaneous Global Functions

_Miscellaneous Global Functions

SimTalk provides the functions listed in the table of contents to the left covering a wide range of applications.

Query the Data Type of a Variable of Data Type Any describes the methods for doing just that.

animation [SimTalk]

Activates or deactivates the animation of **MUs and States** during a simulation run.

Remarks

Calling the function without the parameter returns the current state of the animation.

Calling the function with the parameter sets the state of the animation to activated (`true`) or deactivated (`false`).

Type

Function

Syntax

```
animation → boolean  
animation([Activate:boolean]) → boolean
```

Parameter

The optional parameter *Activate* of data type *boolean* activates (`true`) or deactivates (`false`) the animation of **MUs and States**.

Return Value

The return value has the data type *boolean*.

`true` if the animation of **MUs and States** is activated.

Example

```
animation          // returns if animation is on or off  
animation(true)    // activates animation
```

See also

[MUs and States \[Home ribbon\]](#)



applicationHome [SimTalk]

Returns the folder in which *Plant Simulation* is installed.

Type

Function

Syntax

```
applicationHome → string
```

Return Value

The return value has the data type *string*.

Example

```
print applicationHome // might return C:\Program Files\Siemens\Plant  
Simulation 2606\
```

See also

[Installing Plant Simulation](#)

applicationVersion [SimTalk]

Returns the coded version number of *Plant Simulation*.

Type

Function

Syntax

```
applicationVersion -> integer
```

Return Value

The return value of data type *integer* is composed as follows: VVMMMPPP

- VV is the major version number.
- MMM is the minor version number.
- PPP is the number of the patch level.

Example

```
print applicationVersion // might return 23002008, i.e. version 2302.0008  
print applicationVersion // might return 24004000, i.e. version 2404.0000  
print applicationVersion // might return 26006000, i.e. version 2606.0000
```

See also

[About Plant Simulation](#)

calcBrakingDistance [SimTalk] - general description

Calculates the required braking distance for a given **Speed** and a given **Deceleration**.

Remarks

The braking distance is calculated according to the formula $s = 1/2 v^2/a$.

Note

The function is especially helpful for the *Transporter*.

Type

Function

Syntax

```
calcBrakingDistance(Speed:speed, Deceleration:acceleration) -> length
```

Parameters

You can specify the following parameters:

- The parameter *Speed* of data type *speed* designates the **Speed**.
- The parameter *Deceleration* of data type *acceleration* designates the **Deceleration**.

Example

```
print calcBrakingDistance(100kmh, 10) // returns 138.888888888889m
```

Return Value

The return value has the data type *length*.

See also

[Deceleration \[SimTalk\] - Transporter](#)

[SafetyZones \[SimTalk\]](#)

[calcBrakingDistance \[SimTalk\] - Transporter](#)

calcDroppedPerpendicularFootPoint [SimTalk]

Calculates the foot of the dropped perpendicular of the point P on the line $A1$ - $A2$ and returns it.

Type

Function

Syntax

```
calcDroppedPerpendicularFootPoint(A1:length[3], A2:length[3], P:length[3])  
-> length[3]
```

Parameters

You can specify the following parameters:

- The parameter $A1$ is an *array* of data type *length* with three values. It sets the first point of the line.
- The parameter $A2$ is an *array* of data type *length* with three values. It sets the second point of the line.
- The parameter P is an *array* of data type *length* with three values. It designates the point for which the foot of the dropped perpendicular is calculated.

Return Value

The return value is an *array* with three values of data type *length*.

Example

```
print calcDroppedPerpendicularFootPoint([1,0.1,0.1],[2,0.2,0.4],[5,0.5,1])  
// returns [4.91818181818182, 0.491818181818182, 1.27545454545455]
```

callEvery [SimTalk]

Calls all *Methods* with the designated name in the designated *Frame* and all of its sub-frames.

Remarks

Plant Simulation stops executing the *Method* that contains the *callEvery* call, until all methods called are executed.

Plant Simulation searches the *Frames* for *Methods* using the depth-first strategy. It always immediately executes *Methods* it encounters.

Type

Function

Syntax

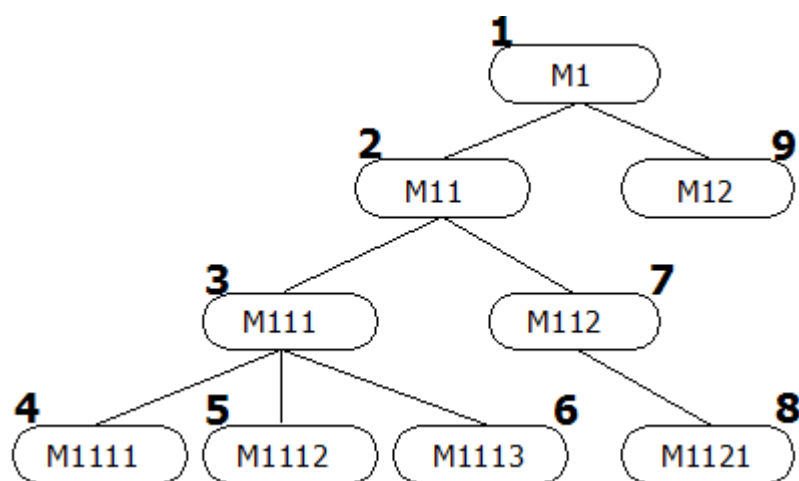
```
callEvery(Frame:path, Method:string[, Parameters:parameters])
```

Parameters

You can specify the following parameters:

- The parameter *Frame* of data type *path* designates the *Frame* within which you want to call all *Methods*.
- The parameter *Method* of data type *string* designates the name of the *Method* you want to call.
- The optional parameters are passed as parameters to the calling *Methods*.

In our example we placed several *Frames* within *Frames* in the model. The numbering M x indicates the sequence the *Frames* were inserted, i.e., *Frame* M 11 was placed before *Frame* M 12, M 111 before M 112, etc. The numbers 1 to 9 indicate the sequence *Plant Simulation* searches for methods with the name specified in *callEvery*. *Plant Simulation* always calls the more recent object first. In our example it calls the *Frames* in this order: 1, 9, 2, 7, 8, 3, 6, 5, 4.



The depth-first strategy starts with the *Frame* entered in Path, and then searches through the hierarchy, using the sequence in which we inserted the *Frames* as a guideline.

The function searches all hierarchy levels for methods with the name of this *Method*. As soon as it finds such a method, it will be called with the designated parameters.

Example

```
callEvery(.frame2, "outputMessage", 999, "here")
```

checkForLicense [SimTalk]

Checks if a user-defined license is available or not.

Type

Function

Syntax

```
checkForLicense(Feature:string, Version:string, PasswordHash:string) →  
integer
```

Parameters

You can specify the following parameters:

- The parameter *Feature* of data type *string* designates the name of the license feature.
- The parameter *Version* of data type *string* designates the requested version number of the license.
- The parameter *PasswordHash* of data type *string* designates the SHA-1-Hash of the password of the license.

Return Value

The return value has the data type *integer*.

The individual values have this meaning:

- 0: The license feature is registered and valid.
- 1: The license feature is not registered.
- 2: The file has an invalid registry data format.
- 3: The SHA-1-Hash is wrong.
- 4: The Host-ID is wrong.
- 5: The registered version of the feature is too old.
- 6: The license has expired.
- 7: The feature does not permit the use with the license type of *Plant Simulation* (Professional / Standard / Runtime /...).

Example

```
if checkForLicense("MyFeature", "1",  
"686483805ac47ca14e03514f7481a7973b401762") = 0  
// The license is available, you can proceed ...  
else  
closeModel  
messageBox("The license 'MyFeature' is unavailable. The model was  
closed.\n", 1, 1)  
end
```

See also

[createLicenseFile \[SimTalk\]](#)

[setRequiredLicense \[SimTalk\]](#)

checkID [SimTalk]

Checks if you can use an expression as the name of an object (true) or not (false).

Type

Function

Syntax

```
checkID(Expression:string) → boolean
```

Parameter

The parameter *Expression* of data type *string* designates the expression. The function considers all keywords and function calls.

Return Value

The return value has the data type *boolean*.

Example

```
print checkID("sin") -- returns false
```

See also

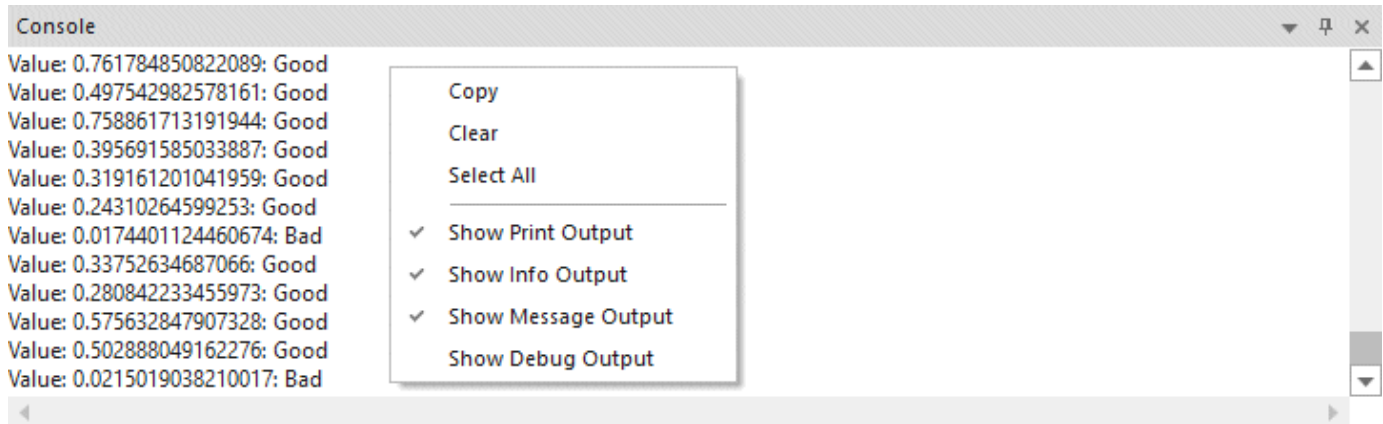
Keywords

clearConsole [SimTalk]

Deletes the contents of the *Console window*.

Remarks

The screenshot shows which data will be deleted.



Type

Function

Syntax

```
clearConsole
```

Example

```
clearConsole
```

See also

[Console](#)

[closeConsole \[SimTalk\]](#)

[openConsole \[SimTalk\]](#)

[setConsoleFilter \[SimTalk\]](#)

[print \[SimTalk\] - output function](#)

Specifying Start Options

clearLogFile [SimTalk]

Deletes the contents of the log file into which *Plant Simulation* wrote all messages which were output to the *Console window*.

Remarks

clearLogFile does not delete the log file itself.

Type

Function

Syntax

```
clearLogFile → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
clearLogFile
```

See also

[Console](#)

[getLogFile \[SimTalk\]](#)

[Specifying Start Options](#)

closeAllWindows [SimTalk]

Closes all open object windows, even those that are minimized, including 3D windows.

Remarks

The optional *boolean* parameter has no effect.

Type

Function

Syntax

```
closeAllWindows([UnusedParameter:boolean])
```

Example

```
closeAllWindows
```

See also

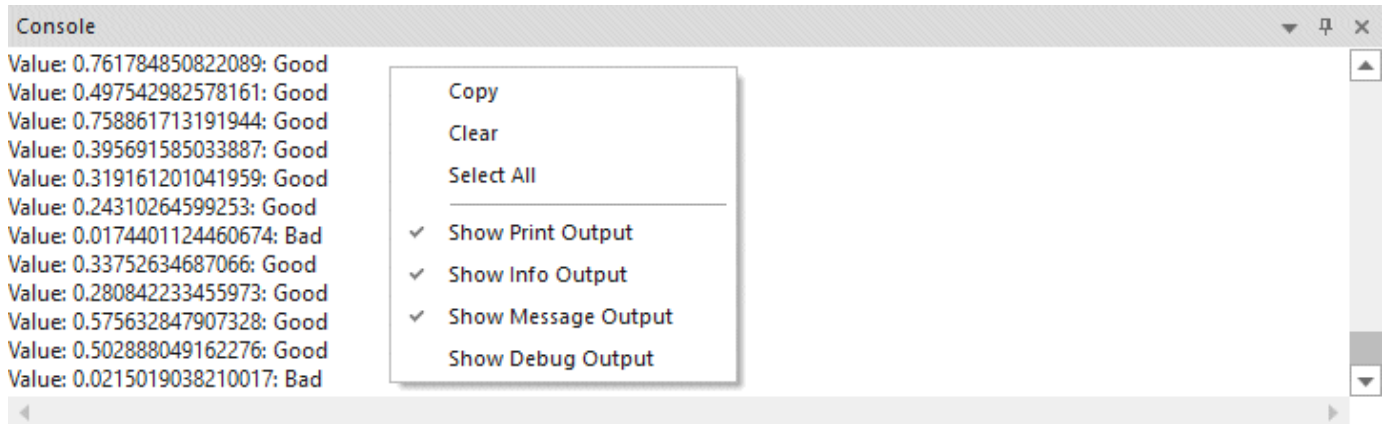
[List of Open Windows](#)

closeConsole [SimTalk]

Closes the open *Console* window.

Remarks

All messages that were output to the *Console* will be lost.



Note

The optional *boolean* parameter has no effect.

Type

Function

Syntax

```
closeConsole([UnusedParameter:boolean]) → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
closeConsole
```

See also

Console

[closeConsole \[SimTalk\]](#)

[clearConsole \[SimTalk\]](#)

[openConsole \[SimTalk\]](#)

[setConsoleFilter \[SimTalk\]](#)

[Specifying Start Options](#)

closeHTMLWindow [SimTalk]

Closes the browser window which you previously opened using the title of the window.

Type

Function

Syntax

```
closeHTMLWindow(CloseHTMLWindow:string) → boolean
```

Parameter

The parameter *CloseHTMLWindow* of data type *string* designates the title of the window you want to close.

Return Value

The return value has the data type *boolean*.

Example

```
closeHTMLWindow("Siemens Corporate Site")  
closeHTMLWindow("Image of the ParallelStation")
```

See also

[openHTMLWindow \[SimTalk\]](#)

closeModel [SimTalk] - global function

Closes the open simulation model without saving it.

Remarks

You can also define a *Method* with the name *onCloseModel* in any folder in the *Class Library*, including the **Basis** folder. This *Method* is being executed when you close the model or when you exit *Plant Simulation*. You can, for example, program clean-up tasks, delete temporary files, etc.

The method *onCloseModel* has to declare the boolean parameter *onExitApplication*. *Plant Simulation* sets this parameter to `false` when *onCloseModel* is called when you close the model. *Plant Simulation* sets this parameter to `true` when *onCloseModel* is called when you exit *Plant Simulation*.

You can also program several *onCloseModel* methods and place them into different folders. *Plant Simulation* then calls all existing *onCloseModel* method classes in the *Class Library* when you close the model or exit the application.

Type

Function

Syntax

```
closeModel → boolean
```

Return Value

The return value has the data type *boolean*.

Plant Simulation returns `false` in these cases and does not close the model:

- When you call the function in sub-routines.
- When you call the function in *user-defined attributes* of data type *method*.
- When you call the function in formulas.

Note

If you want to save your simulation model before you close it, you have to call the function *saveModel* before calling the function *closeModel*.

Note

If the model is closed, all existing *onCloseModel* method classes in the *Class Library*, including the *onCloseModel* method located in the **Basis** folder, are called. Within these methods you can execute clean-up tasks, for example delete temporary files.

Example

```
closeModel
```

See also[Close Model](#)[loadModel \[SimTalk\] - function](#)[saveModel \[SimTalk\]](#)

computeSHA1Hash [SimTalk]

Returns the 40-digit SHA1 hash code of the passed text.

Type

Function

Syntax

```
computeSHA1Hash(Text:string) → string
```

Parameter

The parameter *Text* of data type *string* designates the string for which you would like to know the hash code.

Return Value

The return value has the data type *string*.

Example

```
print computeSHA1Hash("MyText")  
// returns 0e9e68d9402c96044f0f93194f7010bc2e056752
```

See also

[computeSHA3Hash \[SimTalk\]](#)

[strHash \[SimTalk\]](#)

computeSHA3Hash [SimTalk]

Returns the SHA3 hash code of the passed text.

Remarks

computeSHA3Hash uses the SHA3-256-algorithm (NIST: FIPS 202).

Type

Function

Syntax

```
computeSHA3Hash(Text:string) → string
```

Parameter

The parameter *Text* of data type *string* designates the string for which you would like to know the hash code.

Return Value

The return value has the data type *string*.

Example

```
print computeSHA3Hash("MyText")  
// returns 27acf01aa72d499265cc50388f80054e644640586fb04c3165cd4f86d20e02a8
```

See also

[computeSHA1Hash \[SimTalk\]](#)

[strHash \[SimTalk\]](#)

connectAutomatically [SimTalk]

Automatically connects the object designated by <Path> with objects located in its vicinity whose entrances and exits are close together.

Remarks

We added this function because creating an object with *SimTalk* does not automatically connect it.

Plant Simulation proceeds as follows when automatically connecting objects as described below.

Note

Plant Simulation connects objects automatically using 3D-coordinates and 3D-sizes.

- It always uses the first connection possible.
- If a connection exists between the respective object and any other object at a particular side (**Entrance** or **Exit**), *Plant Simulation* does not add a new connection at the same side.
- If two objects are located close to each other, the automatic connection will be created between the respective object and the object with which the connections was established first .
- *Plant Simulation* only establishes an automatic connection, if the bounding boxes overlap in the **Z**-direction.

Note

When automatically connecting objects, *Plant Simulation* uses the first *Connector class* in the **Class Library**, starting at the top.

To change that order, compare [Work with Folders, Frames, Objects in the Class Library](#).

Note

The following applies for automatically connecting the length-oriented objects *Turnplate*, *Turntable*, *AngularConverter*, *Conveyor*, *Converter*, *Track*, and *TwoLaneTrack*:

If a suitable **MU animation path** exists, then *Plant Simulation* uses the start point and the end point of these animation paths for automatically connecting the objects and for displaying *Connectors*. Suitable **MU animation paths** are **Default** or **Cross** for the *Converter*, **Lane A** or **Lane B** for the *TwoLaneTrack* and **Default** for the remaining objects. This applies for **Connect Objects Automatically** and for the method *connectAutomatically*.

Type

Function

Syntax

```
<Path>.connectAutomatically -> boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
MyStation.connectAutomatically
```

See also

[Connect Objects Automatically \[model settings\]](#)

[Connect Objects Automatically \[preferences\]](#)

createCombinations [SimTalk]

Creates all combinations of the specified *Length* of the elements array.

Remarks

If a value is contained multiple times in the elements array, the result also contains multiple combinations.

Type

Function

Syntax

```
createCombinations(Elements:integer[], Length:integer) -> integer[]
```

Parameters

You can specify the following parameters:

- The parameter *Elements* is an array of data type *integer* which designates the elements of the array whose length you would like to create.
- The parameter *Length* of data type *integer* designates the length of the element designated by the parameter *Elements*.

Return Value

The return value is an array of data type *integer*.

The number of generated combinations is limited to 1,000,000.

Examples

```
// Creates all combinations with two elements  
print createCombinations([1,2,3], 2) // returns [1, 2][1, 3][2, 3]
```

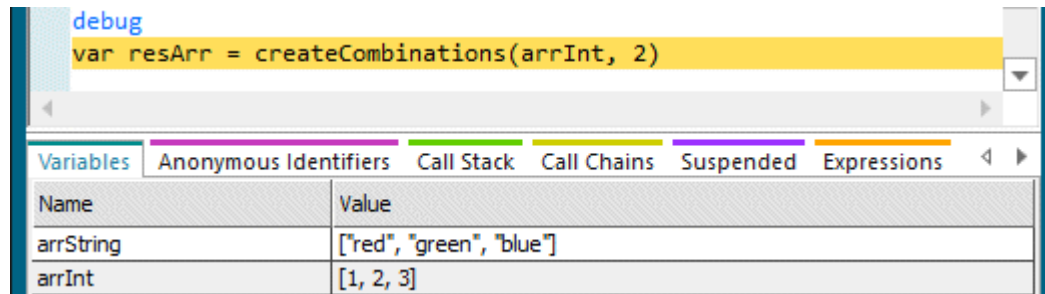
```
// Creates all combinations of two colors  
var arrString:string[]  
var arrInt:integer[]  
  
arrString.append("red")  
arrString.append("green")  
arrString.append("blue")  
  
arrInt.append(1)
```

```
arrInt.append(2)
arrInt.append(3)

var resArr = createCombinations(arrInt, 2)

var colorCombination:string

for var y := 1 to resArr.ydim
  print arrString[resArr[1, y]], ", ", arrString[resArr[2, y]]
next
```



createLicenseFile [SimTalk]

Generates a user-defined license and writes it to a *Windows Registry* file.

Remarks

After generating it, you can then send this file to another person to enter the user-defined license into the *Windows Registry* on the target computer.

You can use the method *setRequiredLicense* to make a library folder request this license. A simulation model/an object containing such a folder can only be loaded if the license is entered into the *Windows Registry* and is valid.

If you want to always permit loading the model or the object, but only want to permit certain actions when a license is available, you can also request the user-defined license in a method by calling the global function *checkForLicense*.

If you deactivate the safety setting **File > Model Settings > General > Prohibit Access to the Computer**, the function *createLicenseFile* can copy data from another folder and write it to the model folder or its sub-folders. The function cannot copy files from the model folder or its sub-folders.

Type

Function

Syntax

```
createLicenseFile(Feature:string, Version:string, HostID:string,
ExpirationDate:string/date, Password:string, FileName:string[,
Restriction:string]) → string
```

Parameters

You can specify the following parameters:

- The parameter *Feature* of data type *string* designates the name of the license feature. The name cannot contain blank spaces.
- The parameter *Version* of data type *string* designates the version number of the license. The registered version has to be greater than or equal to the requested license. The version number cannot contain blank spaces.

If, for example, version 10 of the feature is registered, you can request version 9.5.0 but not vice versa.

- The parameter *HostID* of data type *string* designates the Host ID of the computer for which the license is valid. If you type in an empty string "" or "any", the license is valid on all computers. You can query the valid Host IDs of a computer by starting *Plant Simulation* on the target computer and click **Query Host ID** under **File > Preferences > License**.

Note

Always use the adapter of the physical LAN port for the **Host ID**.

The **Host ID** can be:

- A **MAC address**, which might look like this **300505c9d27c** for example.
- A **Composite Host ID**, which might look like this **COMPOSITE=93FC21BB758E** for example.
- A **Sold-To-ID**, which might look like this **SOLDTO=1234567** for example.

A **Sold-To-ID** has the advantage that the license is not bound to a certain computer but to the *Plant Simulation* license of a customer. This means that the license is valid on each computer that requests a *Plant Simulation* license whose license file has this **Sold-To-ID**. To query the **Sold-To-ID** by starting *Plant Simulation* on the target computer and selecting **File > Help > About Plant Simulation**. The **Host ID** may not contain any blank spaces.

- An **Enterprise Cloud Account ID**, which might look like this **ECA=123456789** for example.
- The parameter *ExpirationDate* of data type *string/date* designates the expiration date until which the license is valid. If you type in an empty string "" or "permanent", the license has no expiration date. The date cannot contain blank spaces.
- The parameter *Password* of data type *string* designates the secret password.
- The parameter *FileName* of data type *string* designates the file name or the file <Path>. The function writes the *Registry* data for the generated license to this file. The file should have the file extension .reg so that the license can be entered into the *Registry* on the target computer by double-clicking it.
- The optional parameter *Restriction* of data type *string* designates the restriction you want to apply to the license. You can only type in "Application", "Runtime", "Simulation", or "Viewer". The license will then only be available if the license type of *Plant Simulation* matches the specified type or a type of lower value. This means that if you restrict the license to "Application", the license is also available in a **Runtime** license and a **Simulation** license of *Plant Simulation*. User-defined licenses are always available in a **Viewer** license.

Return Value

The return value has the data type *string*.

It is the SHA-1-Hash for the password. When you call the function *checkForLicense*, you have to type in this hash. If the hash does not match the hash of the password, the license is considered invalid. We recommend to call the function *checkForLicense* within an encrypted method so that the function call cannot be removed.

It is practically impossible to extract the password from the hash. If you inadvertently give the unencrypted *Method* to somebody else, your password will still be unknown. This is important in case you also used the password somewhere else. As you have to type in the password when generating a license with the function *checkForLicense* no license can be generated using the hash value either.

Example

```
PasswordHash := createLicenseFile("MyLicenseName", "1.0",  
"COMPOSITE=0ADF1C780F29", "", "abc", "C:\TEMP\license.reg")  
  
PasswordHash := createLicenseFile("MyLicenseFeature", "1.0",  
"ECA=111111111", "", "abc", "C:\TEMP\license.reg")
```

See also

[Prohibit Access to the Computer \[model settings\]](#)

[Host IDs](#)

[About Plant Simulation](#)

[checkForLicense \[SimTalk\]](#)

[setRequiredLicense \[SimTalk\]](#)

createPermutations [SimTalk]

Creates all permutations of the specified *Length* of the elements array.

Remarks

If a value is contained multiple times in the elements array, the result also contains multiple permutations.

The number of generated permutations is limited to 1,000,000.

Type

Function

Syntax

```
createPermutations(Elements:integer[], Length:integer) -> integer[]
```

Parameters

You can specify the following parameters:

- The parameter *Elements* is an *array* of data type *integer* which designates the elements of the array whose length you would like to create.
- The parameter *Length* of data type *integer* designates the length of the element designated by the parameter *Elements*.

Return Value

The return value is an array of data type *integer*.

Example

```
// Creates all permutations with two elements  
print createPermutations([1,2,3], 2) // returns [1, 2][1, 3][2, 1][2, 3][3,  
1][3, 2]
```

currentEventCtl [SimTalk]

Returns the *EventController* that controls the current simulation run.

Type

Function

Syntax

```
currentEventCtl → object
```

Return Value

The return value has the data type *object*.

Example

```
if currentEventCtl /= VOID
  print "simulation running in frame", currentEventCtl.location
  print "simulation running in frame", root
end
```

See also

[EventController \[object\]](#)

deg2rad [SimTalk]

Converts the specified angle in degrees into the respective angle in radians.

Type

Function

Syntax

```
deg2rad(AngleInDegrees:real) -> real
```

Parameter

The parameter *AngleInDegrees* of data type *real* designates the angle in degrees which you would like to convert into the angle in radians.

Return Value

The return value has the data type *real*.

Example

```
print deg2rad(45) // returns 0.785398163397448
```

See also

[rad2deg \[SimTalk\]](#)

deleteAllDebugExpressions [SimTalk]

Deletes all debug expressions in all methods.

Type

Function

Syntax

```
deleteAllDebugExpressions
```

Example

```
deleteAllDebugExpressions
```

deleteFile [SimTalk]

Deletes the specified file for good.

Remarks

You can use *deleteFile* to delete any file you do not need any longer.

Type

Function

Syntax

```
deleteFile(FileName:string) -> integer
```

Parameter

The parameter *FileName* of data type *string* designates the file you want to delete.

Return Value

The return value has the data type *integer*.

Is the Windows error code.

Is 0 if no error occurred.

Example

```
deleteFile("MyFile")           // deletes the file named MyFile from the  
installation folder  
deleteFile("D:\Publisher.gif")  // deletes the graphics file from the D  
drive
```

enableFullScreenMode [SimTalk]

Activates (`true`) or deactivates (`false`) full screen mode.

Type

Function

Syntax

```
enableFullScreenMode(Activate:boolean)
```

Parameter

The parameter *Activate* of data type *boolean* activates (`true`) or deactivates (`false`) full screen mode.

Example

```
enableFullScreenMode(true)
```

See also

[Full Screen \[Window ribbon\]](#)

execute [SimTalk] - function

Receives a *string* and interprets it as though it were the source code of a method.

Remarks

The function *execute* accepts all statements. The anonymous identifier *?* refers to the *Method* object that called the *execute* function. The anonymous identifier *@* is received by the calling method.

The function *execute* runs the referenced *Method*. Use *execute* to run *Methods* that you referenced via variables. Let us say you save a *Method* to a variable *obj* of data type *object*, you can then call this *Method* using *obj.execute*.

Note

The syntax, which is used in the argument for the *execute* function, has to be the same as the *Method* syntax which calls the *execute* function.

The semantics of *execute* is as follows:

- If a runtime error occurs while the string method is being executed, there are two possibilities:
- If the setting **Ignore Errors in Formulas** is deactivated, *Plant Simulation* opens the *Debugger*.
- If the setting **Ignore Errors in Formulas** is activated, the function *execute* acts like the function *executeSilent*.
- If no runtime error occurs:
 - If the string method does not have a return value, *execute* returns VOID.

```
var objPath : string := "Track"
var attrName: string := "Length"
var attrVal : length := 3.5
execute(to_str(objPath, ".", attrName, ":", attrVal))
```

- In all other cases, *execute* will return the return value of the *string method*.

```
execute("-> integer; return 42") // returns 42
```

- You can also pass parameters to the *execute statement*.

```
execute("param x, y: integer; print x+y", 1, 2) // outputs 3
```

Type

Function

Syntax

```
execute(SourceCode:string[ ,Parameter:parameter, ...])
```

See also

[executeSilent \[SimTalk\]](#)

Ignore

executePythonFile [SimTalk]

Executes the source code of the specified Python file.

Remarks

You can access the objects in your *Plant Simulation* model in the same way as you would when executing the Python code using a *PythonModule*.

Type

Function

Syntax

```
executePythonFile(File:string) -> boolean
```

Parameter

The parameter *File* of data type *string* designates the Python file that contains the source code that you would like to execute.

Return Value

The return value has the data type *boolean*.

true if no error occurred while executing the source code in the Python file.

false if an error occurred.

Example

```
print executePythonFile("D:\test.py", "TestArg")
```

See also

[Execute External Python Code](#)

executeSilent [SimTalk]

Receives a string and interprets it as though it were the source code of a method.

Remarks

The function *executeSilent* accepts all statements. The anonymous identifier *?* refers to the *Method* object that called the *executeSilent* function. The anonymous identifier *@* is received by the calling method.

The semantics of *executeSilent* is as follows:

- If the string method does not have a return value, *executeSilent* returns VOID.

```
var objPath : string := "Track"
var attrName: string := "Length"
var attrVal : length := 3.5
executeSilent(to_str(objPath, ".", attrName, ":", attrVal))
```

- If the *string* method has a return value, these may look as follows:
 - If the string method contains a syntax error, *executeSilent* returns VOID.

```
executeSilent("-> integer; 1+") // returns VOID
```

- If a runtime error occurs while the string method is executed, *executeSilent* returns the default value of the corresponding type, i.e., the value of a local variable of this type would have at the beginning of the execution of the method. *Plant Simulation* does not open the *Debugger* when a runtime error occurs, but continues to be executed.

Use the function *execute* to make *Plant Simulation* open the *Debugger*, when runtime errors occur. For example, the *executeSilent*-statement below returns false because assigning the keyword "if" to the name of an object causes a runtime error.

```
executeSilent("->boolean; Station.name := \"if\"; return true")
// returns false because of a runtime error
```

- In all other cases, *executeSilent* will return the return value of the string method.

```
executeSilent("->integer; return := 42") // returns 42
```

- You can also pass parameters to the *executeSilent* instruction.

```
executeSilent("param x, y: integer; print x+y", 1, 2) // returns 3
```

The function *executeSilent* executes the referenced *Method*. You can use it to execute *Methods* that you referenced via variables. Let us say you save a *Method* to a variable *obj* of data type *object*, you can now call this *Method* using *obj.executeSilent*.

```
param obj: object, new_name: string
if not executeSilent("param o: object -> boolean; o.name := \"\" + new_name
+ \"\"; return true" obj)
    print "invalid name"
end
```

Be aware that the variant

```
executeSilent("->boolean; " + obj + ".name := \"\" + new_name + \"\"; return
true")
```

does not work, when *obj* points to a method.

The syntax which is used in the argument for the *executeSilent* function has to be the same as the *Method* syntax which calls the *executeSilent* function.

Type

Function

Syntax

```
executeSilent(SourceCode:string[ ,Parameter, ...])
```

See also

[execute \[SimTalk\] - function](#)

[getExecuteSilentError \[SimTalk\]](#)

existsFile [SimTalk]

Returns if a file or folder exists (true) or does not exist (false).

Type

Function

Syntax

```
existsFile(NameOfFileOrFolder:string) → boolean
```

Parameter

The parameter *NameOfFileOrFolder* of data type *string* designates the file or the folder.

Return Value

The return value has the data type *boolean*.

Examples

```
var fileName: string  
fileName := "MyDataTable.tab"  
if not existsFile(fileName)  
    table.writeFile(fileName)  
else  
    print "The file ",fileName," already exists."  
end
```

```
print existsFile("noname.mod")
```

existsMethod [SimTalk]

Returns if a path points to a *Method* object or to a *user-defined attribute* of data type *method* or not.

Type

Function

Syntax

```
existsMethod(NameOfMethod:string) → boolean
```

Parameter

The parameter *NameOfMethod* of data type *string* designates the path of the *Method*.

Return Value

The return value has the data type *boolean*.

true if the *Method* object or the *user-defined attribute* of data type *method* exists.

false if it does not exist.

Example

```
print existsMethod("Station.MethodAttr")
```

existsObject [SimTalk]

Returns if a path to a simulation object is valid or not.

Remarks

The function *existsObject* cannot access the anonymous identifiers **?**, **@**, **self**, and **current**, which are variables of the *Method*.

To check if the object named *Station* exists in the *Frame* **current** use:

```
current.extendPath("Station")
```

Type

Function

Syntax

```
existsObject(NameOfObject:string) → boolean
```

Parameter

The parameter *NameOfObject* of data type *string* designates the path of the object.

Return Value

The return value has the data type *boolean*.

true if the object exists.

false if it does not exist.

Examples

```
print existsObject(".Mus.Part:123")
```

```
if existsObject("ProdFrame")  
  ProdFrame.derive(current,100,200)  
end
```

See also

[_3D.existsObject \[SimTalk\]](#)

[extendPath \[SimTalk\]](#)

exitApplication [SimTalk]

Terminates the simulation run and exits *Plant Simulation*.

Remarks

Plant Simulation does **not save** model data before exiting. Changes to the model or to the simulation data are lost if you do not save the model before, compare the function *saveModel*. *Plant Simulation* does **not** issue a warning or prompt asking you if you really want to exit.

Note

You can also define a *Method* with the name *onCloseModel* in any folder in the *Class Library*, including the **Basis** folder. This *Method* is being executed when you close the model or when you exit *Plant Simulation*. You can, for example, program clean-up tasks, delete temporary files, etc.

The method *onCloseModel* has to declare the boolean parameter `onExitApplication`. *Plant Simulation* sets this parameter to `false` when *onCloseModel* is called when you close the model. *Plant Simulation* sets this parameter to `true` when *onCloseModel* is called when you exit *Plant Simulation*.

You can also program several *onCloseModel* methods and place them into different folders. *Plant Simulation* then calls all existing *onCloseModel* method classes in the *Class Library* when you close the model or exit the application.

Type

Function

Syntax

```
exitApplication
```

Example

```
if EventController.EndTime > 36000
    saveModel("Variation_IIV")
    exitApplication
end
```

See also

[Exit Application](#)

[saveModel \[SimTalk\]](#)

getCallStack [SimTalk]

Returns the active call stack, and writes it to a table.

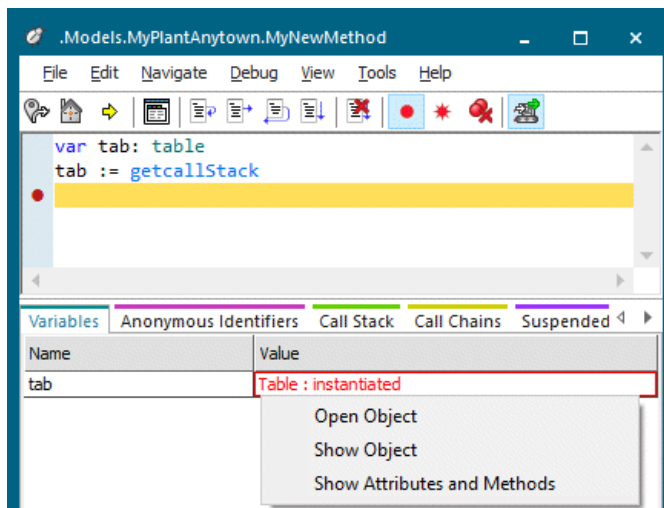
Remarks

The first column is of data type *string* and contains the absolute path of the method.

The second column is of data type *integer* and contains the row number of the executed *SimTalk* instruction.

Note

If *getCallStack* is called in a local *ErrorHandler*, the function returns the call stack of the faulty method and not the call stack of the *ErrorHandler*.



	string	integer
1	.Models.MyPlantAnytown.MyNewMethod	2
2		

Syntax

```
getCallStack → list
```

Return Value

The return value has the data type *list*.

Example

```
var tab: table
tab := getcallStack
```

See also

[Tab Call Stack \[Watch window\]](#)

[Error Handling](#)

[setErrorHandler \[SimTalk\]](#)

getCommandLineArg [SimTalk]

Queries if a command-line argument has been passed to *Plant Simulation* when starting it.

Type

Function

Syntax

```
getCommandLineArg(CommandLineArgument:string, byref Value:string) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *CommandLineArgument* designates the command-line argument that has been passed to *Plant Simulation* when starting it.
- If the parameter *CommandLineArgument* expects a value, you can query this value with the parameter *Value*.

The local variable *Value* is always assigned the value of the next specified command-line argument, compare this example.

```
/NoMessageBox /TrustModels /TestCode true
getCommandLineArg("NoMessageBox", Value) -> Value = '/TrustModels'
getCommandLineArg("TrustModels", Value)  -> Value = '/ TestCode'
getCommandLineArg("TestCode", Value)     -> Value = 'true'
```

Note

If the parameter *CommandLineArgument* does not expect a value, do not specify the parameter *Value* or ignore it.

Return Value

The return value has the data type *boolean*.

true if *Plant Simulation* was started with the specified command-line argument.

false if it was not started with the specified command-line argument.

Example

```
// Suppose we started Plant Simulation with these command-line arguments:  
// PlantSimulation2606.exe /L Runtime /abc 42 /xyz /myfile "C:\Program  
Files\a.txt"  
var arg: string  
if getCommandLineArg("abc", arg)  
    print arg // outputs 42  
end  
if getCommandLineArg("myfile", arg)  
    print arg // outputs C:\Program Files\a.txt  
end  
if getCommandLineArg("xyz", arg)  
    print arg // outputs an empty string  
end
```

See also

[Specifying Start Options](#)

getEpsilon [SimTalk]

Returns the value you set for which *Plant Simulation* considers numerical values to be about equal.

Type

Function

Syntax

```
getEpsilon → real
```

Return Value

The return value has the data type *real*.

Example

```
print getEpsilon // might output 7e-08
```

See also

[Tolerance For About Equal \(~=\) Comparison \[model settings\]](#)

[setEpsilon \[SimTalk\]](#)

getExecuteSilentError [SimTalk]

Returns the error message after calling the function *executeSilent*. *getExecuteSilentError* also returns the error message after a formula has been computed, for example a **formula** in a table or a **Processing Time** of distribution type **Formula**.

Remarks

If an error occurred when executing the function *executeSilent* or while the formula is computed, the function *getExecuteSilentError* returns the error message afterward.

If no error occurred, the function *getExecuteSilentError* returns an empty string. *Plant Simulation* only resets the error when calling *executeSilent* or when computing a formula the next time.

Syntax

```
getExecuteSilentError -> string
```

Return Value

The return value has the data type *string*.

See also

[executeSilent \[SimTalk\]](#)

[Define Processing Times in a Formula](#)

[Formula \[distribution\]](#)

getFileModificationDateTime [SimTalk]

Returns the date and the point in time at which the file was last written.

Type

Function

Syntax

```
getFileModificationDateTime(FilePath:string) → dateTime
```

Parameter

The parameter *FilePath* of data type *string* designates the path of a file for which you would like to know when it was written to last.

You have to enter the entire file path, including the name of the file and the file extension *.spp*.

Return Value

The return value has the data type *dateTime*.

Example

```
print getFileModificationDateTime("D:\MyControls.spp")  
// might, for example, return 2026-05-11 11:24:43.3760
```

getHighResolutionClock [SimTalk]

Returns the number of seconds that passed since opening the model.

Remarks

You can use the function to measure extremely short time spans, for example the time which a *SimTalk* instruction requires.

Type

Function

Syntax

```
getHighResolutionClock → real
```

Return Value

The return value has the data type *real*.

Example

```
var t1, t2: real
t1 := getHighResolutionClock
var x := sin(t1)
t2 := getHighResolutionClock
print "elapsed time: ", (t2-t1)*1000, " milliseconds"

-- might, for example, return elapsed time: 0.028482056222856 milliseconds
```

See also

[processTime \[SimTalk\]](#)

[sysDate \[SimTalk\]](#)

getLibrariesDirectories [SimTalk]

Returns the **Libraries Directories** which you set in the **Preferences**.

Type

Function

Syntax

```
getLibrariesDirectories → string[]
```

Return Value

The return value is an *array* of data type *string*.

Example

```
print getLibrariesDirectories // might return [D:\MeineBibliotheken\  
D:\MyLibraries\]
```

See also

[Libraries Directories](#)

getLibraryFiles [SimTalk]

Returns the file paths to .pslib files and all their versions.

Type

Function

Syntax

```
getLibraryFiles(FolderPath:string) → string[]
```

Parameter

The parameter *FolderPath* of data type *string* is the path of a library folder. The library does not have to exist in the currently loaded model.

Return Value

The return value is an *array* of data type *string*.

It contains all file paths to .pslib files and all versions. The *array* always contains a files path and the associated library version alternating. The *array* is sorted in descending order according to version number.

The returned *array* is empty if no .pslib file is found for the library.

Example

```
var arr: string[] := getLibraryFiles(".Tools.ExperimentManager")
print "Number of library files: ", arr.dim / 2
print "Newest file: ", when arr.dim > 0 then arr[1] else "(not found)"
var i : integer := 1
while i < arr.dim
  print "File: ", arr[i], " Version: ", arr[i+1]
  i += 2
end

-- might, for example, return
Number of library files: 1
Newest file: C:\Plant-Simulation-26.6\Libraries\Tools\Experiment.lib
File: C:\Plant-Simulation-26.6\Libraries\Tools\Experiment.lib Version:
2404.0025
```

getLibraryVersionFromFile [SimTalk]

Returns the version of the library designated by <Path>.

Type

Function

Syntax

```
<Path>.getLibraryVersionFromFile(FilePath:string) → string
```

Parameter

The parameter *FilePath* of data type *string* has to be the path to a library file (.pslib).

Return Value

The return value has the data type *string*.

Example

```
print getLibraryVersionFromFile("C:\Program Files\Siemens\Plant Simulation  
2606\Libraries\Tools\MyLibrary.pslib")
```

See also

[getLibraryInfo \[SimTalk\]](#)

getLogFile [SimTalk]

Gets the log file into which *Plant Simulation* wrote all messages which were output to the *Console*.

Type

Function

Syntax

```
getLogFile → string
```

Return Value

The return value has the data type *string*.

Example

```
getLogFile
```

See also

[clearLogFile \[SimTalk\]](#)

Specifying Start Options

getProfileCallCycles [SimTalk]

Returns a JSON object containing the call cycles that *Profiler* collected provided it is activated.

Type

Function

Syntax

```
getProfileCallCycles([MaxNumCycles:integer]) -> json
```

Parameter

The optional parameter *MaxNumCycles* of data type *integer* restricts the amount of call cycles which *getProfileCallCycles* returns.

To return all call cycles, do not specify the parameter. The amount of returned data can be very large and that the call can take a long time to finish.

Return Value

The return value has the data type *json*.

Example

```
print getProfileCallCycles(10)
```

See also

[Activate Profiler](#)

[profiler \[SimTalk\]](#)

[Activate Profiler](#)

getSeedTable [SimTalk]

Returns the active **Random Number Seed Values table**.

Remarks

The first column contains the random number seed values (*integer*).

The second column contains the comments to the random number seed values (*string*). The number of the row matches the number of the random number stream.

Note

The seed values only apply to the *distribution functions*, such as *z_uniform*, *z_normal*, etc.

Type

Function

Syntax

```
getSeedTable → table
```

Return Value

The return value has the data type *table*.

Example

```
localseedtable := getSeedTable
```

See also

[setSeedTable \[SimTalk\]](#)

[resetRandomNumberStream \[SimTalk\]](#)

[Random Number Seed Values](#)

[Distribution Functions, z_ functions](#)

getStandardColor [SimTalk]

Returns the specified color value from a standard color palette.

Remarks

You can, for example, use *getStandardColor* in the **Entrance Control** of the *Source* to assign different colors to the produced *MUs*.

Type

Function

Syntax

```
getStandardColor(Index:integer) -> integer
```

Parameter

The parameter *Index* of data type *integer* specifies one of 30 color values of a standard color palette.

Return Value

The return value has the data type *integer*.

Example

```
-- Entrance control of a Source object
@._3D.MaterialDiffuseColor = getStandardColor(z_uniform(1,30))
-- Assigns a random color to the created part
```

hideBBL [SimTalk]

Minimizes *Plant Simulation* to an icon in the status area of the taskbar.

Type

Function

Syntax

```
hideBBL(MinimizeWindow:boolean[, HideWindow:boolean]) → boolean
```

Parameters

You can specify the following parameters:

- Setting the parameter *MinimizeWindow* of data type *boolean* to `true` minimizes *Plant Simulation* into the taskbar status area. The application window can be restored by double-clicking the icon in the status area of the taskbar.

Setting the parameter to `false` restores the application window (if it has been minimized to the status area of the taskbar).

Note

The parameter *HideWindow* of data type *boolean* is optional and only relevant, when the parameter *boolean* has the value `true`.

- Setting the parameter *HideWindow* of data type *boolean* to `true` hides the application window without showing an icon status area of the taskbar, so the user cannot restore the application window.

Setting the parameter to `false` shows an icon in the status area of the taskbar, when *Plant Simulation* is minimized in the status area of the taskbar. This is the default.

Return Value

The return value has the data type *boolean*.

Example

```
hideBBL(true) // hides the Plant Simulation window
```

See also

[Specifying Start Options, -hideBBL](#)

isComputerAccessPermitted [SimTalk]

Returns if access to the computer is permitted for the executed *Method* (true) or not (false).

Remarks

Access can be denied for these reasons:

- Access is prohibited under **Model Settings > General > Prohibit Access to the Computer**.
- The executed *Method* belongs to a library and access is prohibited for this library. You find this setting under **Edit Library Information > Prohibit Access to the Computer**.

Type

Function

Syntax

```
isComputerAccessPermitted → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
print isComputerAccessPermitted
```

See also

[Model Settings > Prohibit Access to the Computer \[model settings\]](#)

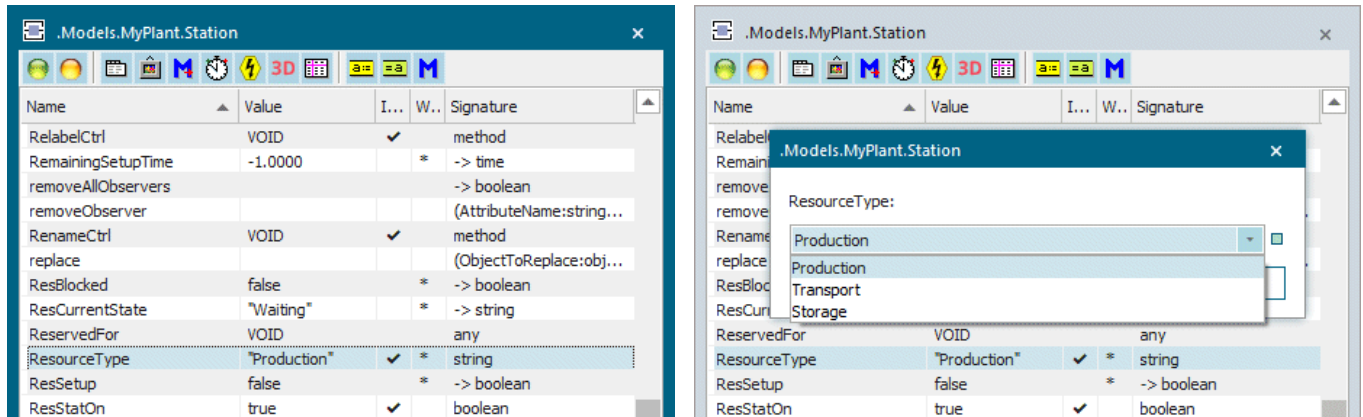
[Edit Library Information > Prohibit Access to the Computer \[Make Library\]](#)

isSet [SimTalk]

Returns if the value of a built-in attribute in *Plant Simulation* is set (true) or not (false).

Remarks

Examples for such attributes are the values of the drop-down lists in the dialogs of the objects. You can also access these settings in the window **Show Attributes and Methods** by double-clicking the respective attribute and selecting a value in the drop-down list.



The attribute **ResourceType** of the material flow objects can, for example, take the values **Production/Produktion**, **Transport/Transport**, or **Storage/Lagerung**, depending on the **model language** and the setting you selected for **Display Down-list Boxes** in the **SimTalk Language of the Model**.

Type

Function

Syntax

```
<Path>.isSet(ValueOfTheAttribute:string) → boolean
```

Parameter

The parameter *ValueOfTheAttribute* of data type *string* designates the value of the attribute.

Return Value

The return value has the data type *boolean*.

Example

```
print MyStation.ResourceType
// the value is Storage, outputs Storage or Lagerung in the Console

print MyStation.ResourceType.isSet("Storage")
// checks, independent of the model language, if the value is Storage or
Lagerung
```

See also

[putValuesIntoTable \[SimTalk\] - function](#)

[Model Language \[model settings\]](#)

[Display Down-list Boxes in the SimTalk Language of the Model](#)

keepWindowsAlwaysOnTop [SimTalk]

Sets if *Plant Simulation* windows will always appear on top of all open windows on the screen or not.

Type

Function

Syntax

```
keepWindowsAlwaysOnTop(WindowInForeground: boolean)
```

Parameter

The parameter *WindowInForeground* of data type *boolean* sets if *Plant Simulation* windows will always appear on top of all open windows on the screen (*true*) or not (*false*).

Example

```
keepWindowsAlwaysOnTop(true)
```

language [SimTalk]

Returns the model language which you selected under **File > Model Settings/Preferences > General > Model Language**.

Remarks

Use *language* to show language-specific messages.

Type

Function

Syntax

```
language → integer
```

Return Value

The return value has the data type *integer*.

0 designates **German**, 1 **English**, 2 **Japanese**, 3 **Chinese**, and 8 designates **Hungarian**.

Example

```
print language
```

See also

[userInterfaceLanguage \[SimTalk\]](#)

[Model Language \[preferences\]](#)

[Model Language \[model settings\]](#)

licenseName [SimTalk]

Returns the name of the active *Plant Simulation* license.

Type

Function

Syntax

```
licenseName → string
```

Return Value

The return value has the data type *string*.

This license	Returns this value
Professional	EMPLANT_PRO
Plant Simulation X Advanced	EMPLANT_PRO
Standard	EMPLANT_STD
Plant Simulation X Standard	EMPLANT_STD
Foundation	EMPLANT_FND
Plant Simulation X Essentials	EMPLANT_FND
Application	EMPLANT_APP
Runtime	EMPLANT_RUN
Plant Simulation X Runtime	EMPLANT_RUN
Research	EMPLANT_RES
Educational	EMPLANT_EDU
Student	EMPLANT_STUDENT
Simulation	EMPLANT_SIM
Viewer	EMPLANT_VIEW

Example

```
print licenseName -- might return EMPLANT_PRO
```

See also

[Start Plant Simulation with Different Kinds of Licenses](#)

[Features of the Individual Licenses](#)

[Plant Simulation License Types](#)

loadModel [SimTalk] - function

Loads the specified simulation model.

Remarks

If you cleared the security setting **File > Model Settings > General > Prohibit Access to the Computer**, the function can copy data from another folder and write it to the model folder or its sub-folders. It cannot copy data from the model folder or its sub-folders.

Type

Function

Syntax

```
loadModel(ModelName:string[, Password:string]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *ModelName* of data type *string* designates the path of the simulation model.
- The optional parameter *Password* of data type *string* designates the password that is used for opening an encrypted model file.

Note

The function only works if no model is loaded at the moment. Typically, you will use the function directly after a call of the function *closeModel* or from external interfaces, such as the *COM interface*.

Return Value

The return value has the data type *boolean*.

Examples

```
closeModel  
loadModel("C:\MySimulationData\myModel.spp", "MyPassword123")
```

```
closeModel  
loadModel("D:\MyModels\MyFolderModel.psfm\$.spp")
```

See also

[Prohibit Access to the Computer \[model settings\]](#)

[Open Model File](#)

[Save As > Folder Model](#)

[closeModel \[SimTalk\] - global function](#)

[saveFolderModel \[SimTalk\]](#)

[saveModel \[SimTalk\]](#)

makePathRelative [SimTalk]

Converts an absolute path to a relative path.

Type

Function

Syntax

```
makePathRelative(Path:string[, StartObject:object]) → string  
makePathRelative(Path:object[, StartObject:object]) → string
```

Parameters

You can specify the following parameters:

- The parameter *Path* of data type *string* or *object* designates the path.
Plant Simulation converts this path into a relative path starting with the *Frame* in which the *Method* is located.
- The optional parameter *StartObject* of data type *object* sets the object from which on *Plant Simulation* starts evaluating the *Path*.

Return Value

The return value has the data type *string*.

Is an empty string "" if the path is not valid.

Example

```
Station.ExitCtrl := makePathRelative(Station.ExitCtrl)
```

See also

[The Absolute Path](#)

[The Relative Path](#)

makeRGBValue [SimTalk]

Generates the RGB color values of the specified color.

Remarks

You can use the RGB values in the attribute *Color* of the objects *Connector*, *Variable*, *Comment*, and *Display* designated by <Path> to efficiently define colors.

Type

Function

Syntax

```
<Path>.makeRGBValue(Red:integer, Green:integer, Blue:integer) → integer
```

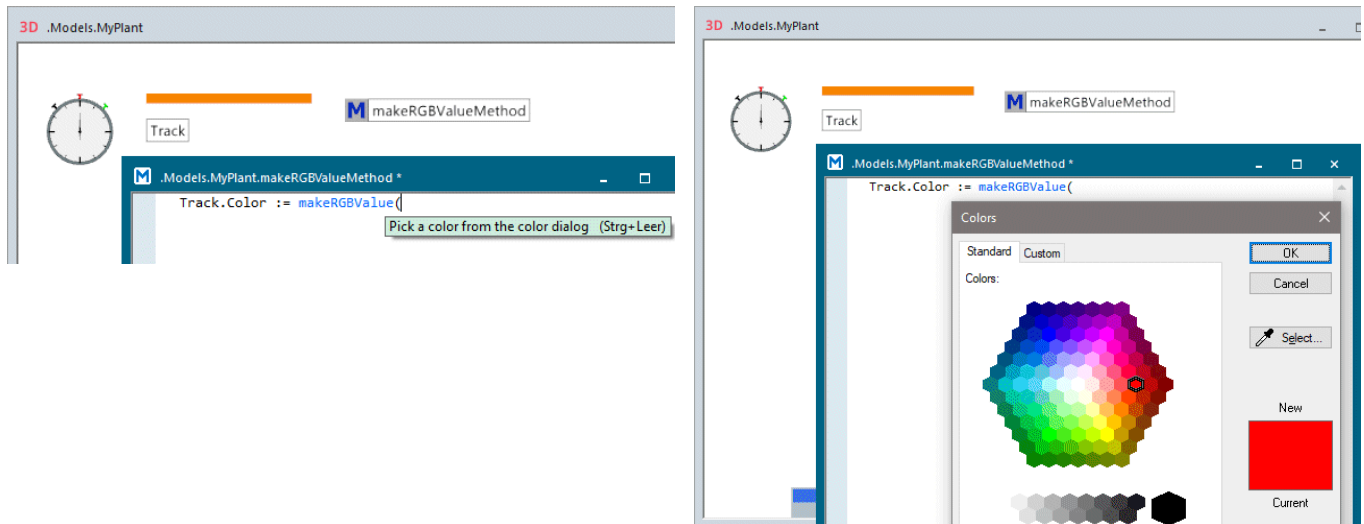
Parameters

You can specify the following parameters:

- The parameter *Red* of data type *integer* designates its **red** component of the color.
- The parameter *Green* of data type *integer* designates its **green** component.
- The parameter *Blue* of data type *integer* designates its **blue** component.

You can specify an integer number between 0 and 255.

Instead of directly typing the values for the color components into the source code of the *Method*, you can also press **Ctrl** and the spacebar to open the dialog **Colors**. Then, select the color in the dialog, and click **OK**. *Plant Simulation* then inserts the color values into the method.



If you already defined a color by setting its color values, *Plant Simulation* recognizes it, selects it in the dialog **Colors**, and transfers its values for red, green, and blue into the source code of your *Method*.

Compare this example:

```
var rgb: integer := makeRGBValue(0,0,255)
```

If you hold down **Ctrl** and press the spacebar in the *Method Editor*, *Plant Simulation* recognizes the color values, and selects the color blue in the dialog **Colors**.

If you then select the color red in the dialog **Colors**, for example, and click **OK**, *Plant Simulation* automatically enters the color values of the color red into the source code of your *Method*, namely:

```
var rgb : integer := makeRGBValue(255,0,0)
```

Return Value

The return value has the data type *integer*.

Example

```
.Models.Model.BackgroundColor.Color := makeRGBValue(110,0,200)
.Models.Model.Connector.Color := makeRGBValue(100,0,200)
.Models.Model.Comment.Color := makeRGBValue(110,0,200)
.Models.Model.Display.Color := makeRGBValue(110,0,200)
.Models.Model.&Variable.Color := makeRGBValue(110,0,200)
```

See also

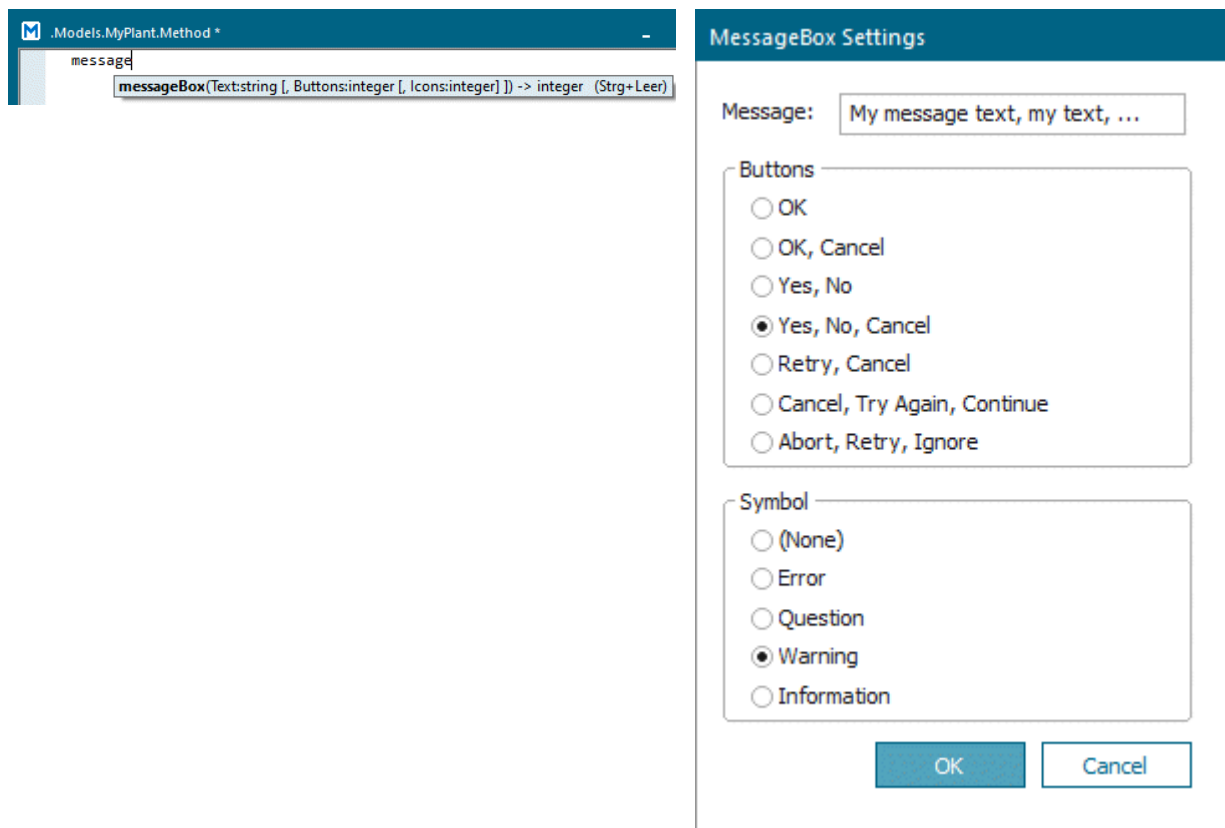
[Color \[SimTalk\] - Connector](#)

messageBox [SimTalk]

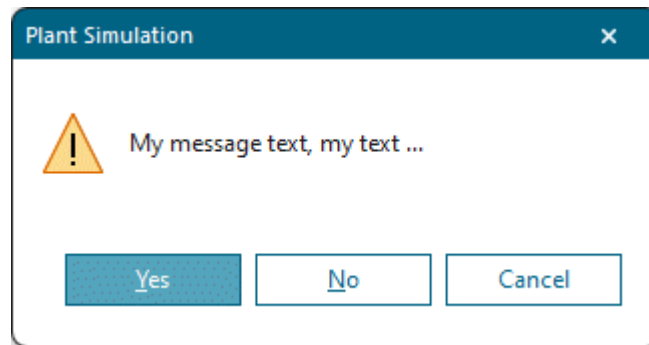
Shows a message box provided by the operating system.

Remarks

Instead of directly typing the text and the numbers into the source code of the method, you can also press **Ctrl** and the spacebar to open the dialog **MessageBox Settings**. Then, type in the message text, select the button combinations, and the symbol, and click **OK**.



In the screenshot below *Plant Simulation* shows the string **My message text, my text ...** which we entered. The number 50 designates the button combination **Yes No Cancel**, and the number 3 shows the yellow triangle with the exclamation mark.



Type

Function

Syntax

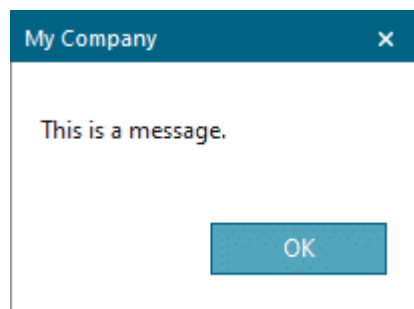
```
messageBox(Text:string[, Buttons:integer[, Icons:integer]]) → integer
```

Parameters

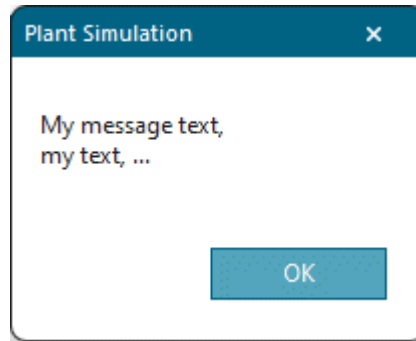
You can specify the following parameters:

- The parameter *Text* of data type *string* designates the text, which you are going to show.

Type in the text, which you want to show in the title bar of the dialog, between two vertical bars, for example `messageBox("|My Company|This is a message.")`.



To insert a line break into your text, type in `strChr(10)`, for example `messageBox("My message text," + strChr(10) + "my text, ...")`.



- The optional parameter *Buttons* of data type *real* designates these buttons:
- 1 the **OK** button
- 3 the button combination **OK, Cancel**
- 10 the button combination **Repeat, Retry, Cancel**
- 48 the button combination **Yes, No**
- 50 the button combination **Yes, No, Cancel**
- 76 the button combination **Abort, Retry, Ignore**

If you do not enter a value for the parameter *Buttons*, the message box only shows the **OK** button.

- The optional parameter *Icons* of data type *real* designates these icons:
- 0 no icon
- 1 an error
- 2 the question mark
- 3 the exclamation mark
- 4 Info

If you do not enter a value for the parameter *Icons*, the message box shows the **exclamation mark**.

Plant Simulation recognizes parameters, which you already defined, and offers this selection in the dialog **MessageBox Parameters**. When you change any of the parameters here, *Plant Simulation* transfers your changed settings to the source code of the *Method* and applies them.

Return Value

The return value has the data type *integer*.

The return value tells you which button/button combinations the user clicked.

Return Value	stands for
1	OK
2	Cancel
4	Ignore
4	Continue
8	Retry
8	Try Again
16	Yes
32	No

Examples

```
switch messageBox("My message text, my text ... ",50, 3)
case 16
    print "Yes"
case 32
    print "No"
else
    print "Cancel"
end
```

```
messageBox("|My Company|This is a message.") // shows My Company in the
title bar
```

modelFile [SimTalk]

Returns the name of the active model file and the folder within which it is located.

Type

Function

Syntax

```
modelFile → string
```

Return Value

The return value has the data type *string*.

Example

```
print modelFile // outputs for example D:\Public\Plant  
Models\MyPlantAnytown.spp
```

See also

[Open Model File](#)

[saveModel \[SimTalk\]](#)